

Week 3 Overview: Multi-Version Concurrency Control

In this part, you will implement MVCC over the LSM engine that you have built in the previous two weeks. We will add timestamp encoding in the keys to maintain multiple versions of a key, and change some part of the engine to ensure old data are either retained or garbage-collected based on whether there are users reading an old version.

The general approach of the MVCC part in this course is inspired and partially based on [BadgerDB](#).

The key of MVCC is to store and access multiple versions of a key in the storage engine. Therefore, we will need to change the key format to `user_key + timestamp (u64)`. And on the user interface side, we will need to have new APIs to help users to gain access to a history version. In summary, we will add a monotonically-increasing timestamp to the key.

In previous parts, we assumed that newer keys are in the upper level of the LSM tree, and older keys are in the lower level of the LSM tree. During compaction, we only keep the latest version of a key if multiple versions are found in multiple levels, and the compaction process will ensure that newer keys will be kept on the upper level by only merging adjacent levels/tiers. In the MVCC implementation, the key with a larger timestamp is the newest key. During compaction, we can only remove the key if no user is accessing an older version of the database. Though not keeping the latest version of key in the upper level may still yield a correct result for the MVCC LSM implementation, in our course, we choose to keep the invariant, and if there are multiple versions of a key, a later version will always appear in a upper level.

Generally, there are two ways of utilizing a storage engine with MVCC support. If the user uses the engine as a standalone component and do not want to manually assign the timestamps of the keys, they will use transaction APIs to store and retrieve data from the storage engine. Timestamps are transparent to the users. The other way is to integrate the storage engine into the system, where the user manages the timestamps by themselves. To compare these two approaches, we can look at the APIs they provide. We use the terminologies of BadgerDB to describe these two usages: the one that hides the timestamp is *un-managed mode*, and the one that gives the user full control is *managed mode*.

Managed Mode APIs

```
get(key, read_timestamp) -> (value, write_timestamp)
scan(key_range, read_timestamp) -> iterator<key, value, write_timestamp>
put/delete/write_batch(key, timestamp)
set_watermark(timestamp) # we will talk about watermarks soon!
```

Un-managed/Normal Mode APIs

```

get(key) -> value
scan(key_range) -> iterator<key, value>
start_transaction() -> txn
txn.put/delete/write_batch(key, timestamp)

```

As you can see, the managed mode APIs requires the user to provide a timestamp when doing the operations. The timestamp may come from some centralized timestamp systems, or from the logs of other systems (i.e., Postgres logical replication log). The user will need to specify a watermark, which is the versions below which the engine can remove.

And for the un-managed APIs, it is the same as what we have implemented before, except that the user will need to write and read data by creating a transaction. When the user creates a transaction, they can gain a consistent state of the database (which is a snapshot). Even if other threads/transactions write data into the database, these data will be invisible to the ongoing transaction. The storage engine manages the timestamps internally and do not expose them to the user.

In this week, we will first spend 3 days doing a refactor on table format and memtables. We will change the key format to key slice and a timestamp. After that, we will implement necessary APIs to provide consistent snapshots and transactions.

We have 7 chapters (days) in this part:

- [Day 1: Timestamp Key Refactor](#). You will change the `key` module to the MVCC one and refactor your system to use key with timestamp.
- [Day 2: Snapshot Read - Memtables and Timestamps](#). You will refactor the memtable and the write path to support multiple version reads/writes.
- [Day 3: Snapshot Read - Transaction API](#). You will implement the transaction API and finish the rest part of read/write path so as to support snapshot reads.
- [Day 4: Watermark and Garbage Collection](#). You will implement the watermark computation algorithm and implement garbage collection at compaction time to remove old versions.
- [Day 5: Transaction and Optimistic Concurrency Control](#). You will create a private workspace for all transactions and commit them in batch so that the modifications of a transaction will not be visible to other transactions.
- [Day 6: Serializable Snapshot Isolation](#). You will implement the OCC serializable checks to ensure the modifications to the database is serializable and abort transactions that violates serializability.
- [Day 7: Compaction Filter](#). At the end of the week, we will generalize the compaction-time garbage collection logic to a compaction filter, that removes data at compaction time as user's requirement.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.